Building Airflow ETL Pipeline

Currency Exchange Rates to MongoDB

Name	M. Abdul Raheem
Roll No.	23L-0588
Section	BCS-7A
Repository	https://github.com/abdul-raheem-fast/currency-exchange-airflow-etl

1. Objective

This project implements a daily Apache Airflow ETL pipeline, created with the Astro CLI. It extracts EUR exchange rates (USD, GBP, JPY) from the Frankfurter API, transforms the response into the required document schema, and loads the records into MongoDB. Mongo Express runs as an additional Docker service for administration. Data moves between tasks using named Airflow XComs.

2. Pipeline Design

Task	Implementation
extract	Calls `https://api.frankfurter.dev/v2/rates?base=EUR&quotes=USD,GBP,JPY` and pushes the response with XCom key `raw_rates`.
transform	Pulls `raw_rates`, builds one document per currency (base, target, rate, date, source, retrieval timestamp) and pushes it with XCom key `transformed_rates`.
load	Pulls `transformed_rates` and inserts the documents into `currency_db.exchange_rates` through MongoHook.

Task dependency: `extract >> transform >> load`. Schedule: `@daily`.

3. Transformed Data Schema

Each document stored in MongoDB follows the required structure (example values shown):

```
{"base_currency": "EUR", "target_currency": "USD", "exchange_rate": 1.15, "rate_date":
"2026-11-04", "source": "Frankfurter API", "retrieved_at": "2026-11-05T07:15:00"}
```

4. Infrastructure

The Astro project adds MongoDB 7.0 and Mongo Express as Docker services in `docker-compose.override.yml`. MongoDB stores data on the persistent `mongo_data` volume. Both services join the `airflow` Docker network, so the Airflow containers reach MongoDB at host `mongo`; Mongo Express is exposed on port 8081. The Dockerfile and both service definitions carry the `maintainer`, `description` and `version` metadata labels.

5. Code Screenshots: Airflow DAG

File: `dags/currency_exchange_etl.py`

5.1 Imports, constants and the extract task (pushes named XCom `raw_rates`)

```python
from datetime import datetime, timezone

import requests
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.mongo.hooks.mongo import MongoHook


API_URL = "https://api.frankfurter.dev/v2/rates"
QUOTES = ["USD", "GBP", "JPY"]
BASE_CURRENCY = "EUR"
MONGO_CONN_ID = "mongo_default"


def extract_rates(**context):
    """Fetch exchange rates from Frankfurter API."""
    response = requests.get(
        API_URL,
        params={"base": BASE_CURRENCY, "quotes": ",".join(QUOTES)},
        timeout=30,
    )
    response.raise_for_status()
    data = response.json()
    if not isinstance(data, list):
        raise ValueError("Frankfurter API returned an unexpected response shape")
    context["ti"].xcom_push(key="raw_rates", value=data)
    print(f"Extracted {len(data)} rate records")
    return data
```

5.2 Transform task (pulls `raw_rates`, pushes `transformed_rates`)

```python
def transform_rates(**context):
    """Reshape raw API data into per-currency documents."""
    raw = context["ti"].xcom_pull(task_ids="extract", key="raw_rates")
    if not isinstance(raw, list):
        raise ValueError("Missing or invalid raw_rates XCom payload")

    retrieved_at = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%S")
    transformed = []
    for record in raw:
        transformed.append(
            {
                "base_currency": record["base"],
                "target_currency": record["quote"],
                "exchange_rate": record["rate"],
                "rate_date": record["date"],
                "source": "Frankfurter API",
                "retrieved_at": retrieved_at,
            }
        )

    context["ti"].xcom_push(key="transformed_rates", value=transformed)
    print(f"Transformed {len(transformed)} records")
    return transformed
```

5.3 Load task (pulls `transformed_rates`, inserts into MongoDB)

```python
def load_to_mongodb(**context):
    """Insert transformed rate documents into MongoDB."""
    records = context["ti"].xcom_pull(task_ids="transform", key="transformed_rates")
    if not isinstance(records, list) or not records:
        raise ValueError("Missing or empty transformed_rates XCom payload")

    client = MongoHook(mongo_conn_id=MONGO_CONN_ID).get_conn()
    try:
        collection = client["currency_db"]["exchange_rates"]
        result = collection.insert_many(records)
        print(f"Inserted {len(result.inserted_ids)} documents into MongoDB")
    finally:
        client.close()
```

5.4 DAG definition and task dependency

```python
with DAG(
    dag_id="currency_exchange_etl",
    description="Daily ETL: Frankfurter API to MongoDB",
    start_date=datetime(2026, 11, 1),
    schedule="@daily",
    catchup=False,
    tags=["etl", "currency", "mongodb"],
) as dag:
    extract = PythonOperator(
        task_id="extract",
        python_callable=extract_rates,
    )

    transform = PythonOperator(
        task_id="transform",
        python_callable=transform_rates,
    )

    load = PythonOperator(
        task_id="load",
        python_callable=load_to_mongodb,
    )

    extract >> transform >> load
```

6. Code Screenshots: Docker Configuration

6.1 Dockerfile with metadata labels

```dockerfile
FROM quay.io/astronomer/astro-runtime:12.7.1

LABEL maintainer="MLOps Assignment 3"
LABEL description="Airflow ETL Pipeline - Currency Exchange Rates to MongoDB"
LABEL version="1.0"
```

6.2 `docker-compose.override.yml`: MongoDB, Mongo Express, network and persistent volume

```yaml
services:
  mongo:
    image: mongo:7.0
    labels:
      maintainer: "MLOps Assignment 3"
      description: "MongoDB for currency exchange rate storage"
      version: "7.0"
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: airflow
      MONGO_INITDB_ROOT_PASSWORD: airflow
    ports:
      - "27017:27017"
    networks:
      - default
      - airflow
    volumes:
      - mongo_data:/data/db

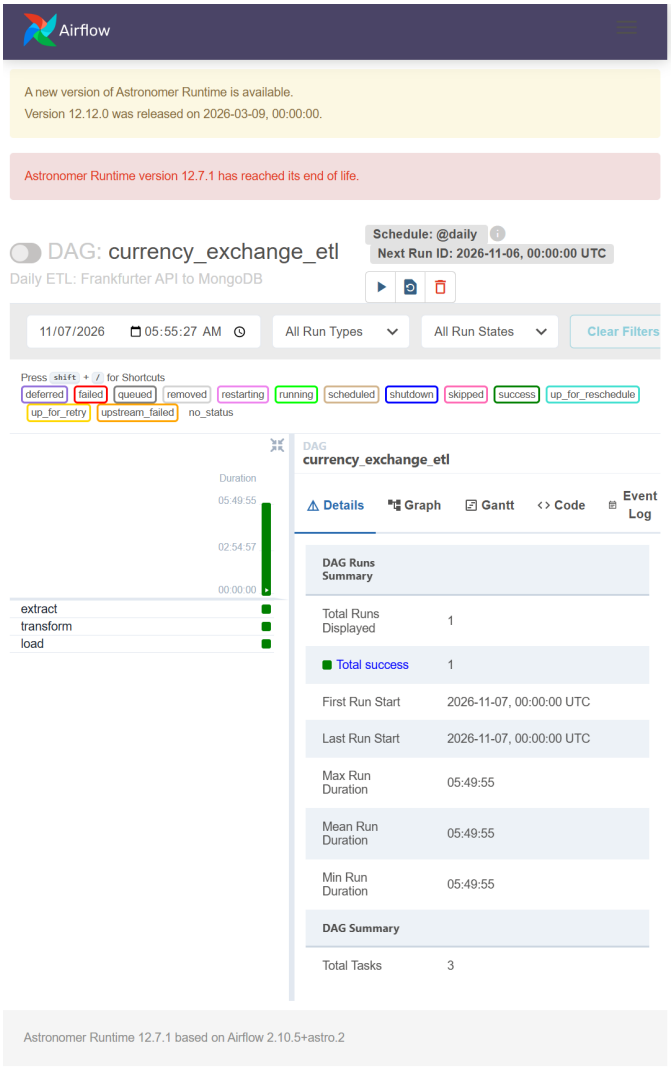
  mongo-express:
    image: mongo-express:1.0
    labels:
      maintainer: "MLOps Assignment 3"
      description: "Mongo Express web UI for MongoDB"
      version: "1.0"
    restart: unless-stopped
    depends_on:
      - mongo
    environment:
      ME_CONFIG_MONGODB_ADMINUSERNAME: airflow
      ME_CONFIG_MONGODB_ADMINPASSWORD: airflow
      ME_CONFIG_MONGODB_URL: mongodb://airflow:airflow@mongo:27017/?authSource=admin
      ME_CONFIG_BASICAUTH: "false"
    ports:
      - "8081:8081"
    networks:
      - default
      - airflow

volumes:
  mongo_data:
    driver: local

networks:
  airflow:
```

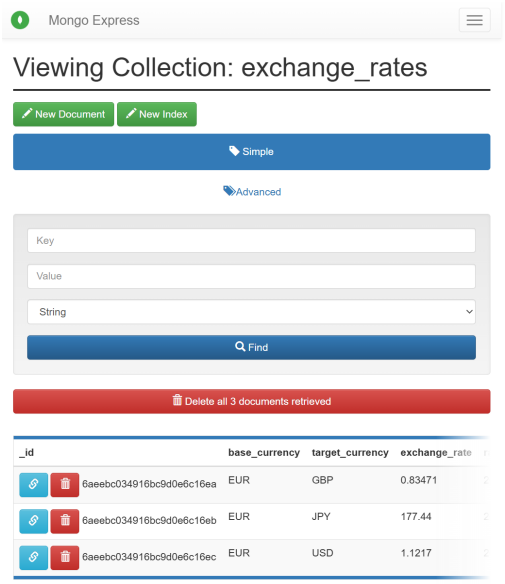
7. Output Screenshot: Airflow

The Airflow grid for DAG `currency_exchange_etl` shows one successful DAG run with all three tasks (extract, transform, load) successful.



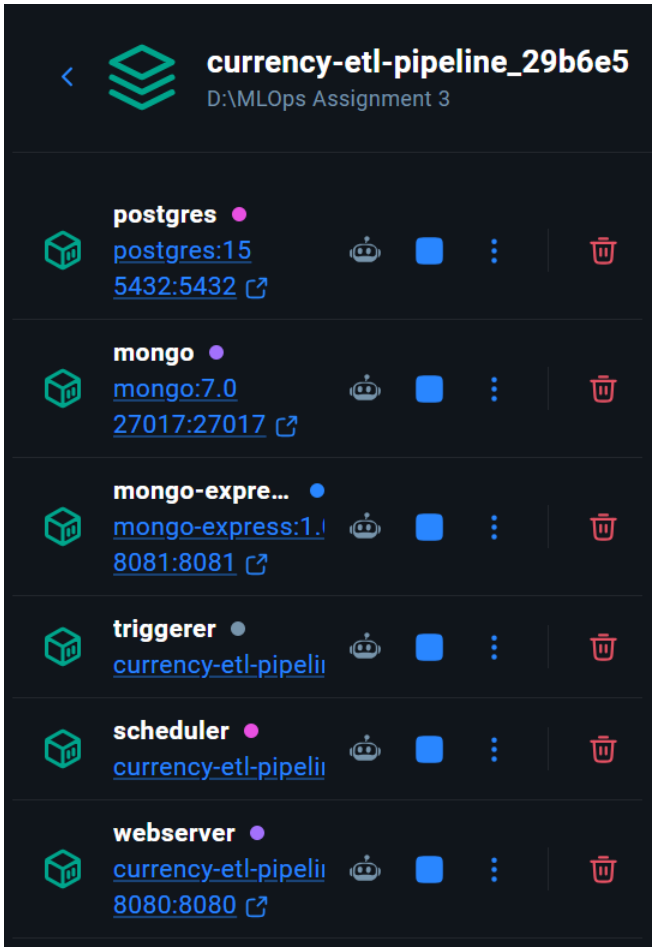
8. Output Screenshot: Mongo Express

Mongo Express shows the `exchange_rates` collection in `currency_db` holding three documents: EUR to GBP, JPY and USD.



9. Output Screenshot: Running Docker Containers

Docker Desktop shows the `currency-etl-pipeline` project with all services running: Airflow (webserver on 8080, scheduler, triggerer), Postgres, `mongo` (MongoDB 7.0 on 27017) and `mongo-express` (on 8081).



10. Requirements Checklist

Requirement	Status
ETL pipeline built with Astro CLI	Completed
Frankfurter API extraction	Completed
Three Airflow tasks (extract, transform, load)	Completed
Named XCom transfer between tasks	Completed
Transformation schema	Completed
Load into MongoDB	Completed
Mongo Express as separate Docker service	Completed
Metadata labels on Docker images	Completed
Docker networking and persistent storage	Completed
Screenshots of code and outputs	Completed
Code pushed to GitHub, link in report	Completed

Repository: https://github.com/abdul-raheem-fast/currency-exchange-airflow-etl